Part 1 Introduction

This lab aims to implement a Reorder Buffer (ROB) with in-order commit support, enhancing the provided I2O2 out-of-order RISC-V processor into an I2OI architecture to ensure that instructions, although executed and written back out of order, are committed to the architectural register file in the correct sequence. The implementation covers ROB organization and control, including the Alloc, Fill, and Commit interfaces, Head/Tail pointer management, pending/valid state tracking, and integration of write-back and bypass mechanisms. Custom test cases are developed to demonstrate errors caused by out-of-order commit and to verify functional correctness after ROB integration. Additionally, this work evaluates how different ROB sizes impact performance, and compares riscvlong and riscvooo in terms of IPC and cycle count across benchmarks, to analyze the trade-offs and performance effects of out-of-order execution and commit control..

Part 2 Design

Describe implementation and why:

This implementation adopts a 16-entry circular Reorder Buffer, using valid, pending, and phys_reg arrays to track the state and destination register of each instruction, while managing allocation and commit through head and tail pointers. A projection-based fullness check (head-after and tail-after comparison) is used to safely determine whether a new entry can be allocated, preventing misjudgment of full conditions when commit and allocation occur in the same cycle. When an execution unit completes, rob_fill_val clears the corresponding pending bit, and if the filled slot is also the current head and ready to commit, a one-cycle registered commit pulse (commit_wen_r) is issued immediately to enforce in-order commit without introducing unnecessary stalls. Registering the commit signal ensures stable timing and alignment with the register file interface, while same-cycle fill-to-head commit and projected fullness logic improve pipeline throughput without violating in-order correctness. Although not functionally mandatory, runtime debug logs and usage counters are included to verify corner cases such as ROB full conditions, allocation rejection, and head-slot bypass commit, ensuring the design behaves as expected during validation.

Part 3 Testing Methodology

Test1.S

```
    // riscv-test1.S
    .section .text
    .global _start
_start:
    addi x2, x0, 1    // x2 = 1
    addi x3, x0, 1    // x3 = 1
    mul  x5, x2, x3
    addi x5, x0, 2
    addi x6, x0, 2     // x6 = expected
    bne  x5, x6, 1f    // if x5 != 2 -> FAIL
    // PASS: csr_status = 1
    addi a0, x0, 1
    csrw 0x15, a0
    j    2f
1:
    addi a0, x0, 2
    csrw 0x15, a0
2:
    ebreak
```

This riscv-test1.S is designed to trigger and reveal the out-of-order commit bug: it first executes a long-latency mul, then overwrites the same destination register x5 using addi. If the processor commits out of order, the mul may commit after the addi, causing x5 to end up with an incorrect value, resulting in a FAIL. With a ROB enforcing in-order commit, the final value of x5 remains correct 2, and the test PASSes.

Test2.S

```
    // riscv-test2.S
    .section .text
    .global _start
_start:
    mul   x1, x0, x0
    addi  x5, x0, 7
    addi  x0, x0, 0   // nop
    addi  x0, x0, 0   // nop
    addi  x0, x0, 0   // nop
    add   x6, x5, x0
    addi  x7, x0, 7
    bne   x6, x7, 1f
    addi  a0, x0, 1
    csrw  0x15, a0
    j     2f
1:
    addi  a0, x0, 2
    csrw  0x15, a0
2:
    ebreak
```

This riscv-test2.S mainly verifies whether the ROB/writeback bypass works correctly: when addi x5, 7 has completed writeback but has not yet committed, the subsequent add x6, x5, x0 should still read the most recent x5 value through bypass. If bypass fails, an outdated value will be read, causing the comparison to fail; if bypass functions correctly, the test passes.

Test3.S

```
   // riscv-test3.S
   .section .text
   .global _start
_start:
   addi  x2, x0, 1        // x2 = 1
   addi  x3, x0, 2        // x3 = 2
   addi  x5, x0, 1        // x5 = 1
   mul   x5, x2, x3       // x5 = 2
   addi  x0, x0, 0        // nop
   addi  x0, x0, 0        // nop
   addi  x6, x0, 2
   bne   x5, x6, 1f
   addi  a0, x0, 1
   csrw  0x15, a0
   j     2f
1:
   addi  a0, x0, 2
   csrw  0x15, a0
2:
```

This riscv-test3.S is designed to verify a WAW (Write-After-Write) scenario: the same destination register x5 is first written by addi (older value) and later overwritten by the slower mul (newer value). Because there is sufficient execution gap between the two writes (and no further writes to x5 afterward), the final committed result will always be the correct value from mul (2), regardless of whether a ROB is used.Confirming that WAW behavior does not corrupt the final result due to differing execution latencies.

Test4.S

```
    // riscv-test4.S
    .section .text
    .global _start
_start:
    addi  x2, x0, 123
    addi  x3, x0, 456
    mul   x1, x2, x3
    addi  x5,  x0, 1
    addi  x6,  x0, 2
    addi  x7,  x0, 3
    addi  x8,  x0, 4
    addi  x9,  x0, 5
    addi  x10, x0, 6
    addi  x11, x0, 7
    addi  x12, x0, 8
    addi  x13, x0, 9
```

```
    addi  x14, x0, 10
    addi  x15, x0, 11
    addi  x16, x0, 12
    addi  x17, x0, 13
    addi  x18, x0, 14
    addi  x19, x0, 15
    addi  x20, x0, 16
    addi  x21, x0, 17
    addi  x22, x0, 18
    addi  x23, x0, 19
    addi  x24, x0, 20
    addi  x25, x0, 21
    addi  x26, x0, 22
    addi  x27, x0, 23
    addi  x28, x0, 24
    addi  x29, x0, 25
    addi  a0, x0, 1
    csrw  0x15, a0
    ebreak
```

This riscv-test4.S demonstrates a case where riscvlong (without a ROB) can achieve higher IPC**.** Except for the initial mul, all remaining instructions are independent addi operations with no data dependencies, allowing riscvlong to execute and retire them rapidly. In contrast, riscvooo (with a ROB) must handle ROB allocation, commit sequencing, and additional control logic, which can introduce stalls or pipeline delays. As a result, riscvlong typically finishes in fewer cycles and with higher IPC.

Test5.S

```
    // riscv-test5.S
    .section .text
    .global _start
_start:
    li    x2, 10007
    addi  x3, x0, 3
    div   x1, x2, x3
    addi  x5,  x0, 1
    addi  x6,  x0, 2
    addi  x7,  x0, 3
    addi  x8,  x0, 4
    addi  x9,  x0, 5
    addi  x10, x0, 6
    addi  x11, x0, 7
    addi  x12, x0, 8
    addi  x13, x0, 9
    addi  x14, x0, 10
    addi  x15, x0, 11
    addi  x16, x0, 12
    addi  x17, x0, 13
    addi  x18, x0, 14
    addi  x19, x0, 15
    addi  x20, x0, 16
    addi  x21, x0, 17
    addi  x22, x0, 18
    addi  x23, x0, 19
    addi  x24, x0, 20
    addi  x25, x0, 21
    addi  x26, x0, 22
    addi  x27, x0, 23
    addi  x28, x0, 24
    addi  x29, x0, 25
    addi  x30, x0, 26
    addi  x31, x0, 27
    addi  a0, x0, 1
    csrw  0x15, a0
    ebreak
```

The purpose of this riscv-test5.S is to fill the ROB entries as much as possible. The test begins with a long-latency div instruction to keep the pipeline busy, followed by a large number of independent addi instructions, forcing the processor to continuously allocate new ROB slots. When the ROB becomes full, the pipeline must stall and wait for commit before issuing more instructions. Therefore, this test is used to analyze how ROB capacity affects performance and stall behavior.

Test1~5 result(have ROB)

```
[ PASSED ] riscv-test1-ooo.out
[ PASSED ] riscv-test2-ooo.out
[ PASSED ] riscv-test3-ooo.out
[ PASSED ] riscv-test4-ooo.out
[ PASSED ] riscv-test5-ooo.out
```

Test1~5 result(not have ROB)

```
[ FAILED ] riscv-test1-ooo.out
[ FAILED ] riscv-test2-ooo.out
[ PASSED ] riscv-test3-ooo.out
[ PASSED ] riscv-test4-ooo.out
[ FAILED ] riscv-test5-ooo.out
```

Without the ROB (Reorder Buffer), riscv-test1, test2, and test5 fail, while test3 and test4 can still pass. The reason is that test1 and test2 involve in-order commit under out-of-order execution and bypassing values that are written back but not yet committed, both of which require ROB support to function correctly. test5 intentionally issues many in-flight instructions to occupy ROB slots, and without a ROB, the processor cannot track multiple pending instructions, so it necessarily fails. In contrast, the WAW case in test3 works because the two writes are separated far enough in time that no conflict occurs even without a ROB, and test4 is only comparing IPC performance, which is unrelated to ROB functionality, so both can pass.

Part 4 *Evalution*

benchmark	core	cycle count	ipc
vvadd	riscvlong	1449	0.735680
vvadd-rand	riscvlong	9788	0.108705
vvadd	riscvooo	1524	0.697507
vvadd-rand	riscvooo	10396	0.102155
masked-filter	riscvlong	10047	0.693341
masked-filter-rand	riscvlong	54408	0.128033
masked-filter	riscvooo	11253	0.618768
masked-filter-rand	riscvooo	60190	0.115667
cmplx-mult	riscvlong	3912	0.754346
cmplx-mult-rand	riscvlong	18751	0.111194
cmplx-mult	riscvooo	4094	0.720078
cmplx-mult-rand	riscvooo	27117	0.108677
bin-search	riscvlong	1541	0.731992
bin-search-rand	riscvlong	5586	0.201396
bin-search	riscvooo	1577	0.713380
bin-search-rand	riscvooo	6211	0.181130

- **When and why one design outperforms the other?**

ANS:The out-of-order core (riscvooo) can issue two independent instructions in parallel, so it outperforms the in-order core (riscvlong) when the code contains sufficient ILP (instruction-level parallelism). However, when dependencies or stalls occur (e.g., load-use hazards or branch delays), the extra control overhead of the scoreboard and ROB causes OOO to lose its advantage, and the simpler riscvlong pipeline can execute faster.

- **Situations where performance differences are minimal**

ANS:When execution is dominated by memory latency or strong data dependencies (such as masked-filter-rand or cmplx-mult-rand), both pipelines stall waiting for data. In these cases, even OOO cannot find independent instructions to issue, so performance becomes similar, and the benefit of dual-issue disappears.

➢ **Which terms in the Iron Law of Processor Performance are affected?**

ANS: Both cores execute the same number of instructions, so instruction count does not change. The OOO design attempts to reduce CPI (or increase IPC) through parallel execution, but its longer critical path and additional control logic increase cycle time. Therefore, OOO mainly affects CPI and cycle time in the Iron Law.

ROB size	Cycle count	ipc
2	71	0.478873
4	38	0.894737
8	37	0.918919
12	38	0.894737
16	37	0.918919

➢ **How performance changes as ROB size increases?**

ANS: With a small ROB (e.g., 2 entries), the core cannot keep enough in-flight instructions to hide latency, resulting in low IPC. As ROB size increases (4 → 8), IPC improves significantly because the core can exploit more ILP. Beyond 8 entries, performance plateaus since the workload does not generate more parallelism than can already fit in the ROB.

➢ **Minimum number of ROB slots needed**

ANS: Based on measurements, ROB=8 achieves the lowest cycle count and highest IPC; increasing it to 12 or 16 does not provide additional benefit. Therefore, the minimum effective ROB size for this lab is 8 entries, as it already captures the maximum useful number of in-flight instructions.

Part 5 *Discussion*

The results show that without a ROB, the processor cannot guarantee in-order commit, causing failures in riscv-test1 (architectural state corruption due to out-of-order commit), test2 (requires bypassing values not yet committed), and test5 (fills ROB capacity). test3 (WAW with sufficient spacing) and test4 (IPC comparison, not dependent on commit correctness) still pass. In performance evaluation, riscv-ooo only outperforms riscvlong when sufficient ILP exists; when execution is dominated by memory latency or data dependencies, the advantage disappears and OOO may even be slower due to control overhead. ROB size analysis shows that IPC improves significantly until 8 entries, after which performance saturates, making 8 slots the optimal ROB size for this design.

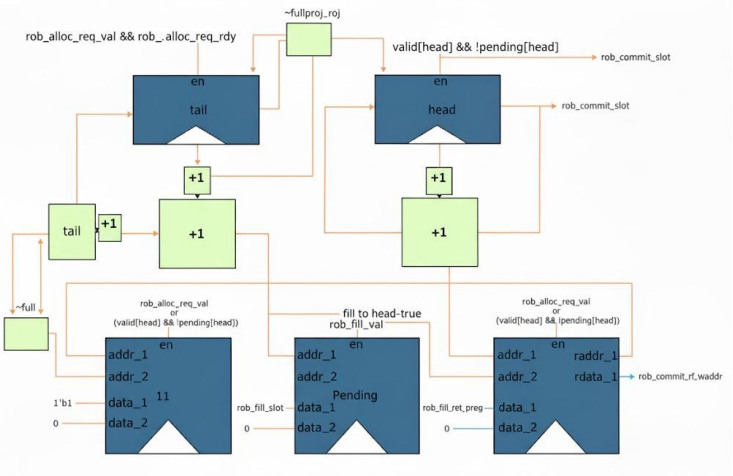

Figure 1: RISCV Reorder Buffer Dataflow